Identify 프로토콜 피어 정보 교환 과정 분석

개요

libp2p-identify는 피어 간 기본 정보를 교환하는 핵심 프로토콜입니다. 이 프로토콜을 통해 피어들은 서로의 신원, 지원하는 프로토콜, 네트워크 주소, 그리고 기타 메타데이터를 안전하게 공유할 수 있습니다.

프로토콜 사양

프로토콜 식별자

- **기본 Identify**: `/ipfs/id/1.0.0`
- **Push Identify**: `/ipfs/id/push/1.0.0`

메시지 구조 (Protobuf)

```
message Identify {
// 프로토콜 호환성을 결정하는 버전 정보
optional  string protocolVersion = 5; // 예: "ipfs/1.0.0"
// 클라이언트 이름과 버전 (HTTP User-Agent와 유사)
optional  string agentVersion = 6; // 예: "rust-libp2p/0.53.0"
// 노드의 공개키 (PeerId 생성에 사용)
optional  bytes publicKey = 1;
// 노드가 리스닝하는 멀티주소들
```

```
repeated bytes listenAddrs = 2;

// 원격 피어가 관찰한 로컬 피어의 주소 (NAT 탐지에 유용)

optional bytes observedAddr = 4;

// 지원하는 프로토콜 목록

repeated string protocols = 3;

// 서명된 피어 레코드 (검증 가능한 주소 광고)

optional bytes signedPeerRecord = 8;

}
```

핵심 구성 요소

1. Behaviour 구조체

```
pub struct Behaviour {

config: Config,

connected: HashMap<PeerId, HashMap<ConnectionId, Multiaddr>>,

our_observed_addresses: HashMap<ConnectionId, Multiaddr>,

events: VecDeque<ToSwarm<Event, InEvent>>,

discovered_peers: PeerCache,

// ... 기타 필드들

}
```

주요 기능:

- 연결된 피어들과의 정보 교환 관리
- 관찰된 주소 추적 및 외부 주소 후보 생성
- 피어 캐시를 통한 주소 발견 최적화

2. Handler 구조체

```
pub struct Handler {
remote_peer_id: PeerId,
active_streams: futures_bounded::FuturesSet<Result<Success, UpgradeError>>,
trigger_next_identify: Delay,
interval: Duration,
local_key: Arc<KeyType>,
// ... 기타 필드들
}
```

주요 기능:

- 연결별 identify 프로토콜 처리
- 주기적 identify 요청 스케줄링
- 인바운드/아웃바운드 스트림 관리

3. Info 구조체

```
pub struct Info {
pub public_key: PublicKey,
pub protocol_version: String,
```

```
pub agent_version: String,

pub listen_addrs: Vec<Multiaddr>,

pub protocols: Vec<StreamProtocol>,

pub observed_addr: Multiaddr,

pub signed_peer_record: Option<SignedEnvelope>,

}
```

정보 교환 과정

1. 연결 설정 후 초기 교환

```
피어 A 피어 B

| |

|--- TCP/QUIC 연결 설정 -------->|

| |

|<-- multistream-select -------->|

| (/ipfs/id/1.0.0) |

| |

|--- Identify 요청 ------------->|

| |

|<-- Identify 응답 -------------|

| (Info 구조체) |

| |
```

```
|<-- Identify 요청 --------------|
| |
|--- Identify 응답 ------------->|
| (Info 구조체) |
```

2. 주기적 정보 업데이트

기본적으로 5분마다 identify 정보를 재교환합니다:

```
// 기본 설정
interval: Duration::from_secs(5 * 60)
```

3. Push 메커니즘

정보가 변경되었을 때 능동적으로 업데이트를 전송:

```
// 주소 변경 시 자동 push (설정에 따라)
push_listen_addr_updates: bool

// 수동 push 트리거
pub fn push<I>(&mut self, peers: I)
where
I: IntoIterator<Item = PeerId>
```

상세 동작 분석

1. 연결 핸들러 생성

새로운 연결이 설정되면 각 연결에 대해 Handler가 생성됩니다:

```
fn handle_established_inbound_connection(
&mut  self,
_: ConnectionId,
peer: PeerId,
_: &Multiaddr,
remote_addr: &Multiaddr,
) -> Result<THandler<Self>, ConnectionDenied> {
Ok(Handler::new(
self.config.interval,
peer,
self.config.local_key.clone(),
self.config.protocol_version.clone(),
self.config.agent_version.clone(),
remote_addr.clone(),
self.all_addresses(),
))
}
```

2. 프로토콜 네고시에이션

Handler는 두 가지 프로토콜을 지원합니다:

```
fn listen_protocol(&self) -> SubstreamProtocol<Self::InboundProtocol> {
SubstreamProtocol::new(
SelectUpgrade::new(
ReadyUpgrade::new(PROTOCOL_NAME), // /ipfs/id/1.0.0
ReadyUpgrade::new(PUSH_PROTOCOL_NAME), // /ipfs/id/push/1.0.0
),
(),
)
}
```

3. 정보 구성

로컬 피어 정보는 다음과 같이 구성됩니다:

```
fn build_info(&mut  self) -> Info {
let signed_envelope = match  self.local_key.as_ref() {
KeyType::PublicKey(_) => None,
KeyType::Keypair { keypair, .. } => {
libp2p_core::PeerRecord::new(
keypair,
Vec::from_iter(self.external_addresses.iter().cloned()),
)
.ok()
.map(|r| r.into_signed_envelope())
```

```
}
};
Info {
public_key: self.local_key.public_key().clone(),
protocol_version: self.protocol_version.clone(),
agent_version: self.agent_version.clone(),
listen_addrs: Vec::from_iter(self.external_addresses.iter().cloned()),
protocols: Vec::from_iter(self.local_supported_protocols.iter().cloned()),
observed_addr: self.observed_addr.clone(),
signed_peer_record: signed_envelope,
}
}
```

4. 메시지 직렬화 및 전송

```
pub(crate) async  fn send_identify<T>(io: T, info: Info) -> Result<Info, Upgrade
where
T: AsyncWrite + Unpin,
{
let message = proto::Identify {
agentVersion: Some(info.agent_version.clone()),
protocolVersion: Some(info.protocol_version.clone()),
publicKey: Some(info.public_key.encode_protobuf()),
```

```
listenAddrs: info.listen_addrs.iter().map(|addr| addr.to_vec()).collect(),

observedAddr: Some(info.observed_addr.to_vec()),

protocols: info.protocols.iter().map(|p| p.to_string()).collect(),

signedPeerRecord: info.signed_peer_record

.clone()

.map(|r| r.into_protobuf_encoding()),

};



let  mut framed_io = FramedWrite::new(

io,

quick_protobuf_codec::Codec::<proto::Identify>::new(MAX_MESSAGE_SIZE_BYTES),

);



framed_io.send(message).await?;

framed_io.close().await?;

Ok(info)

}
```

5. 수신 및 검증

수신된 정보는 다음과 같이 검증됩니다:

```
fn handle_incoming_info(&mut  self, info: &Info) -> bool {

let derived_peer_id = info.public_key.to_peer_id();
```

```
if  self.remote_peer_id != derived_peer_id {
return  false; // 공개키가 피어 ID와 일치하지 않으면 거부
}

self.remote_info.replace(info.clone());
self.update_supported_protocols_for_remote(info);
true
}
```

이벤트 처리

1. 생성되는 이벤트들

```
pub  enum Event {
/// 피어로부터 식별 정보를 수신
Received {
connection_id: ConnectionId,
peer_id: PeerId,
info: Info,
},
/// 로컬 노드 정보를 피어에게 전송
Sent {
connection_id: ConnectionId,
```

```
peer_id: PeerId,
},
/// 로컬 노드 정보를 능동적으로 push
Pushed {
connection_id: ConnectionId,
peer_id: PeerId,
info: Info,
},
/// 식별 과정에서 오류 발생
Error {
connection_id: ConnectionId,
peer_id: PeerId,
error: StreamUpgradeError<UpgradeError>,
},
}
```

2. 외부 주소 발견

관찰된 주소를 통해 외부 주소 후보를 생성합니다:

```
fn emit_new_external_addr_candidate_event(
&mut  self,
connection_id: ConnectionId,
observed: &Multiaddr,
```

```
) {
if  self.outbound_connections_with_ephemeral_port.contains(&connection_id) {
// 주소 변환 적용 (TCP 포트 재사용 없는 경우)
let translated_addresses = {
let  mut addrs: Vec<_> = self
.listen_addresses
.iter()
.filter_map(|server| {
if (is_tcp_addr(server) && is_tcp_addr(observed))
|| (is_quic_addr(server, true) && is_quic_addr(observed, true))
|| (is_quic_addr(server, false) && is_quic_addr(observed, false))
{
_address_translation(server, observed)
} else {
None
}
})
.collect();
addrs.sort_unstable();
addrs.dedup();
addrs
};
```

```
if translated_addresses.is_empty() {
self.events.push_back(ToSwarm::NewExternalAddrCandidate(observed.clone()));
} else {
for addr in translated_addresses {
self.events.push_back(ToSwarm::NewExternalAddrCandidate(addr));
}
}
return;
}

self.events.push_back(ToSwarm::NewExternalAddrCandidate(observed.clone()));
}
```

보안 고려사항

1. 공개키 검증

수신된 정보의 공개키가 연결된 피어의 ID와 일치하는지 확인:

```
let derived_peer_id = info.public_key.to_peer_id();
if  self.remote_peer_id != derived_peer_id {
tracing::warn!(
%self.remote_peer_id,
?info.public_key,
```

```
derived_peer_id=%derived_peer_id,
"Discarding received identify message as public key does not match remote peer I
);
return  false;
}
```

2. 서명된 피어 레코드

검증 가능한 주소 광고를 위한 서명된 레코드:

```
let (listen_addrs, signed_envelope) = msg
.signedPeerRecord
.and_then(|b| {
let envelope = SignedEnvelope::from_protobuf_encoding(b.as_ref()).ok()?;
let peer_record = PeerRecord::from_signed_envelope(envelope).ok()?;
(peer_record.peer_id() == identify_public_key.to_peer_id()).then_some((
peer_record.addresses().to_vec(),
Some(peer_record.into_signed_envelope()),
))
})
.unwrap_or_else(|| (parse_listen_addrs(msg.listenAddrs), None));
```

3. 멀티주소 검증

수신된 주소가 피어 ID와 일치하는지 확인:

```
fn multiaddr_matches_peer_id(addr: &Multiaddr, peer_id: &PeerId) -> bool {
let last_component = addr.iter().last();
if  let Some(multiaddr::Protocol::P2p(multi_addr_peer_id)) = last_component {
return multi_addr_peer_id == *peer_id;
}
true
}

// 사용 예시
info.listen_addrs.retain(|addr| multiaddr_matches_peer_id(addr, &peer_id));
```

설정 옵션

1. 기본 설정

```
pub  struct Config {
protocol_version: String, // 프로토콜 버전
local_key: Arc<KeyType>, // 로컬 키 (공개키 또는 키페어)
agent_version: String, // 에이전트 버전
interval: Duration, // 주기적 식별 간격 (기본: 5분)
push_listen_addr_updates: bool, // 주소 변경 시 자동 push (기본: false)
cache_size: usize, // 피어 캐시 크기 (기본: 100)
```

```
hide_listen_addrs: bool, // 리스닝 주소 숨김 (기본: false)
}
```

2. 설정 예시

```
let config = identify::Config::new(
"my-app/1.0.0".to_string(),
keypair.public(),
)
.with_agent_version("my-client/0.1.0".to_string())
.with_interval(Duration::from_secs(300))
.with_push_listen_addr_updates(true)
.with_cache_size(200)
.with_hide_listen_addrs(false);

let behaviour = identify::Behaviour::new(config);
```

실제 사용 예시

1. 기본 사용법

```
use libp2p::{identify, swarm::NetworkBehaviour};
```

```
#[derive(NetworkBehaviour)]
struct MyBehaviour {
identify: identify::Behaviour,
// 다른 프로토콜들...
}


// 설정
let behaviour = MyBehaviour {
identify: identify::Behaviour::new(
identify::Config::new(
"my-protocol/1.0.0".to_string(),
keypair.public(),
)
),
};
```

2. 이벤트 처리

```
match swarm.next().await {
SwarmEvent::Behaviour(MyBehaviourEvent::Identify(identify::Event::Received {
peer_id,
info,
..
```

```
})) => {
println!("피어 {}로부터 정보 수신:", peer_id);
println!(" 프로토콜 버전: {}", info.protocol_version);
println!(" 에이전트: {}", info.agent_version);
println!(" 지원 프로토콜: {:?}", info.protocols);
println!(" 리스닝 주소: {:?}", info.listen_addrs);
println!(" 관찰된 주소: {}", info.observed_addr);
// 외부 주소로 관찰된 주소 추가
swarm.add_external_address(info.observed_addr);
}
SwarmEvent::NewExternalAddrCandidate { address } => {
println!("새로운 외부 주소 후보: {}", address);
}
// 기타 이벤트 처리...
}
```

3. 능동적 Push

```
// 특정 피어들에게 정보 push
let peers_to_notify = vec![peer1, peer2, peer3];
swarm.behaviour_mut().identify.push(peers_to_notify);
```

성능 및 최적화

1. 스트림 제한

```
const MAX_CONCURRENT_STREAMS_PER_CONNECTION: usize = 10;

const STREAM_TIMEOUT: Duration = Duration::from_secs(60);
```

2. 메시지 크기 제한

```
const MAX_MESSAGE_SIZE_BYTES: usize = 4096;
```

3. 피어 캐시

발견된 피어 주소를 LRU 캐시에 저장하여 재연결 시 활용:

```
struct PeerCache(Option<PeerAddresses>);


impl PeerCache {

fn enabled(size: NonZeroUsize) -> Self {

Self(Some(PeerAddresses::new(size)))

}

fn get(&mut  self, peer: &PeerId) -> Vec<Multiaddr> {

if  let Some(cache) = self.0.as_mut() {

cache.get(peer).collect()

} else {

Vec::new()
```

```
}

}

}
```

결론

libp2p-identify 프로토콜은 P2P 네트워크에서 피어 간 기본 정보 교환을 담당하는 핵심 구성 요소입니다. 이 프로토콜을 통해:

1. **피어 신원 확인**: 공개키를 통한 안전한 피어 식별
2. **네트워크 발견**: 리스닝 주소와 관찰된 주소를 통한 네트워크 토폴로지 파악
3. **프로토콜 호환성**: 지원하는 프로토콜 목록을 통한 기능 협상
4. **NAT 탐지**: 관찰된 주소를 통한 네트워크 환경 파악
5. **동적 업데이트**: Push 메커니즘을 통한 실시간 정보 동기화

이러한 기능들이 결합되어 libp2p 네트워크의 자동 구성과 효율적인 피어 관리를 가능하게 합니다.